

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

C++ 基础数学：从取模到组合数

Core Number Theory Algorithms in C++

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 14 日

目录

1. 取模运算
2. 高精度
3. 素数判定
4. 约数
5. 质因数分解
6. 快速幂

目录

⑦ 最大公约数与最小公倍数

⑧ 组合数

⑨ 总结

目录

1 取模运算
2 高精度
3 素数判定
4 约数
5 质因数分解
6 快速幂

目录

7 最大公约数与最小公倍数

8 组合数

9 总结

取模运算：三条可用性质

- **加法**：$(a+b) \bmod m = \big((a \bmod m) + (b \bmod m)\big) \bmod m$
- **减法**：$(a-b) \bmod m = \big((a \bmod m) - (b \bmod m) + m\big) \bmod m$
- **乘法**：$(a \times b) \bmod m = \big((a \bmod m) \times (b \bmod m)\big) \bmod m$
- **除法**：没有对应性质，不能直接拆开

减法先加模数再取模；乘法注意 `long long` 溢出。

代码：取模的加、减、乘


```cpp
const long long MOD = 1e9 + 7;

long long addMod(long long a, long long b) {
    return (a + b) % MOD;
}

long long subMod(long long a, long long b) {
    return ((a - b) % MOD + MOD) % MOD;   // 先加 MOD, 避免出现负数 *
}

long long mulMod(long long a, long long b) {
    return a * b % MOD;                   // a, b 小于 MOD 时安全
}

// 模数超过 1e9 时, 乘法需要 __int128
long long mulModBig(long long a, long long b, long long mod) {
    return (long long)((__int128)a * b % mod);
}
```

目录

1 取模运算

2 高精度

3 素数判定

4 约数

5 质因数分解

6 快速幂

目录

7 最大公约数与最小公倍数

8 组合数

9 总结

高精度：存储约定

- 用 `vector` 容器逐位存储，下标 0 存个位，即逆序存储
- 逆序的好处：进位、借位都只需要向数组尾部追加
- 输入：从字符串倒着读入，字符减去字符零的 ASCII 值
- 输出：从高位到低位倒着打印
- 统一去掉前导零：删除尾部所有 0，直到只剩一位

下标小的一端是个位，所有进位与借位都朝数组尾部走。

代码：高精度加法


```cpp
// A, B 均为逆序存储的非负整数
vector<int> add(vector<int> A, vector<int> B) {
    if (A.size() < B.size()) swap(A, B);
    vector<int> C;
    int t = 0;                                  // t 为进位
    for (int i = 0; i < (int)A.size(); i++) {
        t += A[i];
        if (i < (int)B.size()) t += B[i];
        C.push_back(t % 10);
        t /= 10;                                // 进位传递 *
    }
    if (t) C.push_back(t);
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}
```

逐位相加并维护进位，最后处理最高位的溢出。

代码：高精度减法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```cpp
// 判断 A >= B (均为逆序存储)
bool geq(vector<int>& A, vector<int>& B) {
    if (A.size() != B.size()) return A.size() > B.size();
    for (int i = (int)A.size() - 1; i >= 0; i--)
        if (A[i] != B[i]) return A[i] > B[i];
    return true;
}

// 要求 A >= B
vector<int> sub(vector<int>& A, vector<int>& B) {
    vector<int> C;
    int t = 0;                              // t 为借位
    for (int i = 0; i < (int)A.size(); i++) {
        t = A[i] - t;
        if (i < (int)B.size()) t -= B[i];
        C.push_back((t + 10) % 10);
        t = (t < 0) ? 1 : 0;                // 产生借位 *
    }
```

代码：高精度减法


```
19      while (C.size() > 1 && C.back() == 0) C.pop_back();
20      return C;
21  }
```

代码：高精度乘法（大数乘大数）


```cpp
// A, B 均为逆序存储的非负整数
// 竖式乘法：C[i + j] 累加 A[i] * B[j]
vector<int> mul(vector<int>& A, vector<int>& B) {
    vector<int> C(A.size() + B.size() + 1, 0);
    for (int i = 0; i < (int)A.size(); i++)
        for (int j = 0; j < (int)B.size(); j++)
            C[i + j] += A[i] * B[j];      // 按位累加 *
    // 统一处理每一位的进位
    for (int i = 0; i < (int)C.size() - 1; i++) {
        C[i + 1] += C[i] / 10;
        C[i] %= 10;
    }
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}
```

复杂度 $O(nm)$，是竖式乘法的直接模拟。

代码：大数除以小数


```cpp
// A 是大数（逆序存储），b 是普通整数，r 返回余数
vector<int> divSmall(vector<int>& A, int b, int& r) {
    vector<int> C;
    r = 0;
    for (int i = (int)A.size() - 1; i >= 0; i--) {
        r = r * 10 + A[i];                  // 拉下一位，拼到余数后面
        C.push_back(r / b);                 // 当前位的商 *
        r %= b;                             // 更新余数 *
    }
    reverse(C.begin(), C.end());            // 翻转成逆序存储
    while (C.size() > 1 && C.back() == 0) C.pop_back();
    return C;
}
```

从最高位开始处理，余数逐位传递；只需 $O(n)$。

代码：大数除以大数（辅助函数）


```cpp
// 判断 A >= B
bool geq(vector<int>& A, vector<int>& B) {
    if (A.size() != B.size()) return A.size() > B.size();
    for (int i = (int)A.size() - 1; i >= 0; i--)
        if (A[i] != B[i]) return A[i] > B[i];
    return true;
}

// 大数减法，要求 A >= B
vector<int> sub(vector<int>& A, vector<int>& B) {
    vector<int> C;
    int t = 0;
    for (int i = 0; i < (int)A.size(); i++) {
        t = A[i] - t;
        if (i < (int)B.size()) t -= B[i];
        C.push_back((t + 10) % 10);
        t = (t < 0) ? 1 : 0;
    }
```

代码：大数除以大数（辅助函数）


```
19      while (C.size() > 1 && C.back() == 0) C.pop_back();
20      return C;
21  }
```

代码：大数除以大数（主函数）


```cpp
// A = Q * B + R, Q 为商, R 为余数, 均为逆序存储
vector<int> divBig(vector<int>& A, vector<int>& B, vector<int>& R) {
    vector<int> Q;
    R.clear();
    for (int i = (int)A.size() - 1; i >= 0; i--) {
        R.insert(R.begin(), A[i]);          // 拉下一位, 插到高位
        while (R.size() > 1 && R.back() == 0)
            R.pop_back();                   // 去掉前导零
        int cnt = 0;
        while (geq(R, B)) {                 // 逐次减 B, 统计减了几次 *
            R = sub(R, B);
            cnt++;
        }
        Q.push_back(cnt);                   // 当前位的商 *
    }
    reverse(Q.begin(), Q.end());
    while (Q.size() > 1 && Q.back() == 0) Q.pop_back();
    return Q;
```

代码：大数除以大数（主函数）


```
19 }
```

竖式除法：每一位的商在 0 到 9 之间，内层最多减 9 次。

目录

1 取模运算

2 高精度

3 素数判定

4 约数

5 质因数分解

6 快速幂

目录

7 最大公约数与最小公倍数

8 组合数

9 总结

代码：试除法判定素数

- 只需枚举到 $\sqrt{n}$，循环条件写成 $i \le n/i$ 可避免 $i \times i$ 溢出
- 单次判定复杂度 $O(\sqrt{n})$，适合 n 不超过 10^{12}

```
1 bool isPrime(long long n) {
2     if (n < 2) return false;
3     for (long long i = 2; i <= n / i; i++)
4         if (n % i == 0) return false;
5     return true;
6 }
```

代码：埃氏筛


```cpp
const int N = 1e7 + 10;
bool st[N];
vector<int> primes;

// 筛出 [1, n] 内所有质数
void sieve(int n) {
    for (int i = 2; i <= n; i++) {
        if (st[i]) continue;                // i 已被更小的质数标记过
        primes.push_back(i);
        for (long long j = (long long)i * i; j <= n; j += i)
            st[j] = true;                   // 从 i*i 开始标记 *
    }
}
```

复杂度 $O(n \log \log n)$；从 i^2 开始是因为更小的倍数已被筛过。

代码：线性筛（欧拉筛）


```cpp
void linearSieve(int n) {
    for (int i = 2; i <= n; i++) {
        if (!st[i]) primes.push_back(i);
        for (int p : primes) {
            if ((long long)p * i > n) break;
            st[p * i] = true;              // 每个合数只被最小质因子筛一次 *
            if (i % p == 0) break;
        }
    }
}
```

整除时立刻退出，是保证整体复杂度为 $O(n)$ 的关键。

目录

1 取模运算

2 高精度

3 素数判定

4 约数

5 质因数分解

6 快速幂

目录

7 最大公约数与最小公倍数

8 组合数

9 总结

约数：核心公式

设 $n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$。

目标	公式
约数个数	$(a_1+1)(a_2+1)\cdots(a_k+1)$
约数之和	$(1+p_1+\cdots+p_1^{a_1})\cdots(1+p_k+\cdots+p_k^{a_k})$

约数之和用等比数列累加实现，避免除法。

代码：试除法枚举所有约数


```cpp
vector<int> getDivisors(int n) {
    vector<int> res;
    for (int i = 1; i <= n / i; i++) {
        if (n % i == 0) {
            res.push_back(i);
            if (i != n / i)                    // 完全平方数只加一次 *
                res.push_back(n / i);
        }
    }
    sort(res.begin(), res.end());
    return res;
}
```

复杂度 $O(\sqrt{n})$；约数成对出现，只需枚举到 $\sqrt{n}$。

代码：约数个数与约数之和


```cpp
// f 为质因数分解结果：{质数, 指数}
long long countDivisors(vector<pair<int,int>>& f) {
    long long res = 1;
    for (auto& e : f) res *= (e.second + 1);   // 乘法原理 *
    return res;
}

long long sumDivisors(vector<pair<int,int>>& f) {
    long long res = 1;
    for (auto& e : f) {
        long long p = e.first, a = e.second;
        long long t = 1, pw = 1;                    // t = 1 + p + ... + p^a
        for (int i = 0; i < a; i++) {
            pw *= p;
            t += pw;
        }
        res *= t;
    }
```

代码：约数个数与约数之和


```
19      return res;
20  }
```

目录

1 取模运算

2 高精度

3 素数判定

4 约数

5 质因数分解

6 快速幂

目录

7 最大公约数与最小公倍数

8 组合数

9 总结

代码：试除法质因数分解


```cpp
vector<pair<int,int>> factorize(int n) {
    vector<pair<int,int>> res;
    for (int i = 2; i <= n / i; i++) {
        if (n % i == 0) {
            int cnt = 0;
            while (n % i == 0) {
                n /= i;
                cnt++;
            }
            res.push_back({i, cnt});
        }
    }
    if (n > 1) res.push_back({n, 1});   // 剩下的一定是质数 *
    return res;
}
```

循环结束后若 n 仍大于 1，它本身就是最后一个质因子。

代码：最小质因子筛与快速分解


```cpp
const int N = 1e7 + 10;
int spf[N];                                  // smallest prime factor

void buildSpf(int n) {
    for (int i = 2; i <= n; i++) {
        if (spf[i]) continue;
        for (int j = i; j <= n; j += i)
            if (!spf[j]) spf[j] = i;         // 只记录最小质因子 *
    }
}

// 单次分解复杂度 O(log n)
vector<pair<int,int>> factorizeFast(int x) {
    vector<pair<int,int>> res;
    while (x > 1) {
        int p = spf[x], cnt = 0;
        while (x % p == 0) {
            x /= p;
```

代码：最小质因子筛与快速分解

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

```
19              cnt++;
20          }
21          res.push_back({p, cnt});
22      }
23      return res;
24  }
```

目录

1 取模运算

2 高精度

3 素数判定

4 约数

5 质因数分解

6 快速幂

目录

7 最大公约数与最小公倍数

8 组合数

9 总结

快速幂：二进制拆分思想

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

目标：计算 $a^b \bmod m$，**其中** b **可达** 10^{18}。

以 3^{13} **为例，**$13 = 8 + 4 + 1$，**于是** $3^{13} = 3^8 \times 3^4 \times 3^1$。

二进制位	1	1	0	1
当前底数	3^8	3^4	3^2	3^1
是否累乘	是	是	否	是

每一步把底数平方，遇到 1 就累乘，循环次数 $O(\log b)$。

代码：快速幂（迭代版）


```cpp
// 计算 a^b mod m
long long qpow(long long a, long long b, long long m) {
    long long res = 1 % m;                  // m == 1 时也能正确返回 0 *
    a %= m;
    while (b > 0) {
        if (b & 1) res = res * a % m;       // 当前位为 1，累乘 *
        a = a * a % m;                      // 底数平方 *
        b >>= 1;                            // 指数右移一位
    }
    return res;
}
```

若模数大于 10^9，乘法需改用 __int128。

快速幂的典型用途

- 大指数取模：直接循环会超时，快速幂可降到 $O(\log b)$
- 幂运算优化：任何形如 $a^b \bmod m$ 的计算都可以套用
- 与其他算法配合：是许多数论算法的基础模板

快速幂是整个数论章节里复用率最高的模板，务必写熟。

目录

1 取模运算

2 高精度

3 素数判定

4 约数

5 质因数分解

6 快速幂

目录

7 最大公约数与最小公倍数

8 组合数

9 总结

欧几里得算法：原理

- **核心结论**：$\gcd(a, b) = \gcd(b, a \bmod b)$
- **边界条件**：$b = 0$ **时**，$\gcd(a, 0) = a$
- **时间复杂度**：$O(\log \min(a, b))$
- C++17 **起标准库提供** `std::gcd` **与** `std::lcm`

$\gcd$ **与** lcm **的关系**：$\gcd(a, b) \times \mathrm{lcm}(a, b) = a \times b$。

代码：gcd 与 lcm


```cpp
// 递归版：一行写法 *
long long gcd(long long a, long long b) {
    return b == 0 ? a : gcd(b, a % b);
}

// 迭代版：避免递归深度问题
long long gcdIter(long long a, long long b) {
    while (b) {
        long long t = a % b;
        a = b;
        b = t;
    }
    return a;
}

// 最小公倍数：先除后乘，防止溢出 *
long long lcm(long long a, long long b) {
    return a / gcd(a, b) * b;
```

代码：gcd 与 lcm


```
19 }
```

lcm 必须先除后乘；先乘后除会在大数输入下溢出。

目录

1 取模运算

2 高精度

3 素数判定

4 约数

5 质因数分解

6 快速幂

目录

7 最大公约数与最小公倍数

8 组合数

9 总结

组合数：杨辉三角递推

递推公式：

$$\binom{n}{m} = \binom{n-1}{m-1} + \binom{n-1}{m}$$

n \m	0	1	2	3
0	1			
1	1	1		
2	1	2	1	
3	1	3	3	1

每一格等于上方两格之和，边界全是 1。

代码：杨辉三角预处理


```cpp
const int N = 2010;
long long C[N][N];

// 预处理 [0, n] 范围内所有组合数
void initComb(int n, long long MOD) {
    for (int i = 0; i <= n; i++) {
        C[i][0] = C[i][i] = 1;                  // 边界全是 1 *
        for (int j = 1; j < i; j++)
            C[i][j] = (C[i-1][j-1] + C[i-1][j]) % MOD;   // 递推 *
    }
}

// 查询时直接取 C[n][m]
long long comb(int n, int m) {
    if (m < 0 || m > n) return 0;               // 越界返回 0 *
    return C[n][m];
}
```

代码：杨辉三角预处理

优点是模数不必是质数，缺点是只能承受 n 不超过 2000 的规模。

目录

1 取模运算

2 高精度

3 素数判定

4 约数

5 质因数分解

6 快速幂

目录

⑦ 最大公约数与最小公倍数

⑧ 组合数

⑨ 总结

总结：复杂度速查

算法	复杂度	典型适用规模
试除法判素	$O(\sqrt{n})$	n 不超过 10^{12}，单次询问
埃氏筛	$O(n \log \log n)$	n 不超过 10^7
线性筛	$O(n)$	n 不超过 10^8
试除法分解	$O(\sqrt{n})$	n 不超过 10^{12}，单次分解
快速幂	$O(\log b)$	b 不超过 10^{18}
$\gcd$ / lcm	$O(\log n)$	任意范围
杨辉三角组合数	$O(n^2)$ 预处理	n 不超过 2000

总结：六个高频易错点

- 减法取模忘记先加模数，导致出现负数
- 高精度减法忘记判断被减数是否大于等于减数
- 大数乘大数时，忘记在末尾统一处理进位
- lcm 先乘后除，结果溢出
- 快速幂漏掉 `res = 1 % m`，模数为 1 时出错
- 线性筛忘记整除立即退出，退化成非线性的

模板写熟之后，剩下的就是把边界条件想清楚。

附录：整数溢出与常用常量

类型	大致范围
`int`	约 2.1×10^9
`unsigned int`	约 4.3×10^9
`long long`	约 9.2×10^{18}
`__int128`	约 1.7×10^{38}

常用模数：$10^9 + 7$、$10^9 + 9$、998244353。

附录：常见提交陷阱

- 大数组开在全局，避免栈溢出
- 数组大小比数据范围多开 10 个，防止越界
- `memset` 只能按字节赋值，只适合 0 / -1 / 0x3f
- 多组测试数据时，全局数组与计数器要重新清空
- `vector` 作为参数时，加引用可避免整体拷贝
- 筛法数组用 `bool` 即可，无需 `int`